Software Engineering Department
Faculty of Computing
Universiti Teknologi Malaysia

SCSE2233 — Software Design and Architecture

Semester 2, 2025/2026 — Section 3

Software Design Description (SDD)

Online Food Delivery System

Assignment 2 — Design Patterns & Implementation-Ready Architecture

Prepared by: Group Hub (Group 11)

Lecturer: Muhammad Khatib Syarbini

Team Members:

Team Members Name	Student ID
Mohammed Mohsen Mohammed Alsakkaf	A23CS4026
Anas Ahmed Hisham	A24CS4006
Imtiaj Ahmed	A24CS4017

1. Frontispiece

1.1 Overview

This document is Version 2.0 of the Software Design Description (SDD) for the Online Food Delivery System. It extends the structural foundation established in Assignment 1 (Version 1.0 — case study, use case model, microservices architectural style, and per-module UML) into an implementation-ready design. The current version injects standardized GoF design patterns into each member-owned module, supplies concrete Java code samples translated directly from the UML, and adds the database (Information), interface, and algorithm viewpoints required by the IEEE 1016 design framework.

1.2 Issuing Team

Issued by Group Hub (Group 11), School of Computing, for the SCSE2233 Software Design and Architecture course. The team owns the end-to-end design of the Online Food Delivery System case study, partitioned into three independently designed modules.

1.3 Authorship / Project Team Members

Name	Matric No.	Role	Module	Task Status
Mohammed Mohsen Mohammed Alsakkaf	A23CS4026	Lead Architect & Project Manager	Module 1 — User Auth & Restaurant Browsing	**Complete**
Anas Ahmed Hisham	A24CS4006	System Analyst	Module 2 — Shopping Cart & Order Placement	**Complete**
Imtiaj Ahmed	A24CS4017	Interface Designer	Module 3 — Payment Processing & Delivery Tracking	**Complete**

1.4 Revision History

Revision No.	Issue Date	Details of Revision
1.0	May 2026	Initial submission (Assignment 1): case study selection, use case diagram, use case specifications, sequence diagrams, microservices architectural style, and per-module UML class diagrams.
2.0	June 2026	Assignment 2: added Information viewpoint (ERD + data dictionary), Interface viewpoint (wireframes), Logical viewpoint with one creational, one structural, and one behavioral design pattern per module,

Revision No.	Issue Date	Details of Revision
		pattern-aware class and sequence diagrams, Algorithm viewpoint (activity & state diagrams), Java code samples, and coherence-validation tables.

Table of Contents

1. Frontispiece **2**
1.1 Overview 2
1.2 Issuing Team 2
1.3 Authorship / Project Team Members 2
1.4 Revision History 2
2. Introduction **5**
2.1 Purpose 5
2.2 Scope 5
2.3 Context 5
2.4 Summary 5
3. References **7**
4. Glossary **8**
5. Design Body **9**
5.1 Design Stakeholders and Their Concerns 9
5.2 Context Viewpoint (Use Case Diagram) 10
5.2.1 Design Concerns 10
5.2.2 Design View (Use Case Diagram) 10
Actors 10
Use Case Specifications 10
5.3 Structure Viewpoint (Architecture Style — Assignment 1) 13
5.3.1 Design Concerns 13
5.3.2 Design View (Component / Architecture Diagram) 13
5.4 Information Viewpoint (Database Design) 14
5.4.1 Design Concerns 14
5.4.2 Design View (ERD) 14
Data Dictionary 14
Entity Relationship Diagram (ERD) 15
5.5 Interface Viewpoint (Interface Design) 16
5.5.1 Design Concerns 16
User Interfaces 16
Hardware Interfaces 16
Software Interfaces 16
Communication Interfaces 16
5.5.2 Design View (Wireframes) 16

5.6 Logical Viewpoint (Creational, Structural & Behavioral Patterns — Assignment 2)....18
5.6.1 Design Concerns.... 18
5.6.2 Module 1 — User Auth & Restaurant Browsing | Mohammed Alsakkaf.... 19
(a) Creational Pattern — Singleton (ConfigurationManager).... 19
(b) Behavioral Pattern — Strategy (AuthenticationStrategy)....19
(c) Structural Pattern — Proxy (RestaurantCacheProxy)....20
Coherence Validation — Module 1.... 20
5.6.3 Module 2 — Shopping Cart & Order Placement | Anas Ahmed Hisham....22
(a) Creational Pattern — Builder (OrderBuilder)....22
(b) Structural Pattern — Facade (CheckoutFacade)....22
(c) Behavioral Pattern — State (OrderState)....23
Coherence Validation — Module 2.... 23
5.6.4 Module 3 — Payment Processing & Delivery Tracking | Imtiaj Ahmed.... 24
(a) Creational Pattern — Factory Method (PaymentGatewayFactory)....24
(b) Structural Pattern — Adapter (StripeGatewayAdapter).... 24
(c) Behavioral Pattern — Observer (TrackingService).... 25
Coherence Validation — Module 3.... 25
5.7 Interaction Viewpoint (Sequence Diagrams by Module).... 27
5.7.1 Design Concerns.... 27
5.7.2 Design View (Sequence Diagrams).... 27
5.8 Algorithm Viewpoint (Construction Design for Complex Processing)....29
5.8.1 Design Concerns.... 29
5.8.2 Design View (Activity & State Diagrams)....29
6. User Interface Design....30
7. Appendices....31
7.1 Appendix A — Module Assignment & Pattern Matrix....31
7.2 Appendix B — Group Work Contributions....31
7.3 Appendix C — Final Quality Assurance Checklist....31

2. Introduction

The following sections of this Software Design Description (SDD) provide the complete design of the Online Food Delivery System. The document follows the IEEE 1016 viewpoint model, organizing the design as a set of complementary views, each addressing a specific set of stakeholder concerns.

2.1 Purpose

This SDD describes the detailed software design of the Online Food Delivery System, evolving the Assignment 1 architecture into an implementation-ready blueprint. Its purpose is twofold: to specify how the system's classes, data, interfaces, and interactions are organized, and to demonstrate the disciplined application of object-oriented design patterns that make the system extensible, maintainable, and ready for coding.

The intended audience is:

- **Software designers and developers** — who implement the modules from the class diagrams, sequence diagrams, and code samples;
- **Software architects** — who verify that the design honours the microservices style and non-functional requirements;
- **Quality assurance engineers** — who derive test cases from the use cases, sequences, and activity diagrams;
- **Course evaluator / lecturer** — who assesses architectural coherence and correct pattern application.

2.2 Scope

The software product specified is the **Online Food Delivery System** — a distributed, multi-actor platform that lets customers discover restaurants, build a cart, place and pay for orders, and track delivery in real time. It is comparable in scope to Grab Food, Foodpanda, and Uber Eats.

The system will: authenticate users; allow restaurant and menu browsing with search and filtering; manage shopping carts and order placement; process payments securely through an external gateway; and stream live delivery tracking. The system will not implement the restaurant-side kitchen-management application, the driver dispatch-optimization engine, or third-party loyalty programs; these are treated as external systems or out of scope. The design is consistent with the system requirements and the microservices architectural style selected in Assignment 1.

2.3 Context

This SDD is to be used by the project's technical and managerial stakeholders. It serves as a guide for the designer in the selection, organization, and presentation of design information, and as a reference contract between design and implementation. It is intended to be used in conjunction with the Assignment 1 Software Architecture report (Version 1.0), from which the use case model and architectural style are carried forward unchanged.

2.4 Summary

The remainder of this document is organized as follows. Section 3 lists the references. Section 4 defines the glossary. Section 5 — the Design Body — is the core of the SDD and presents eight design views: design stakeholders and concerns (5.1); the Context viewpoint with the use case model (5.2); the Structure viewpoint with the microservices architecture (5.3); the Information viewpoint with the ERD and data dictionary (5.4); the Interface viewpoint (5.5); the Logical viewpoint, which introduces the creational, structural, and behavioral design patterns and their code (5.6); the Interaction viewpoint with pattern-aware sequence diagrams (5.7); and the Algorithm viewpoint (5.8). Section 6 consolidates the user interface design, and Section 7 holds the appendices.

3. References

The following documents are referenced throughout this SDD:

[1] IEEE Std. 1016-2009, IEEE Standard for Information Technology — Systems Design — Software Design Descriptions. IEEE Computer Society.

[2] IEEE Std. 830-1998, IEEE Recommended Practice for Software Requirements Specifications. IEEE Computer Society.

[3] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

[4] Newman, S. (2021). Building Microservices, 2nd ed. O'Reilly Media.

[5] Group Hub (2026). Software Architecture Design — Assignment 1 Report (Version 1.0): Online Food Delivery System. SCSE2233.

[6] UML Diagrams Examples. Retrieved from https://www.uml-diagrams.org/index-examples.html

4. Glossary

The following terms, acronyms, and abbreviations are used in this SDD:

Term / Acronym	Definition
Adapter	Structural design pattern that converts the interface of a class (e.g., an external API) into another interface clients expect.
Builder	Creational design pattern that constructs a complex object step by step, separating construction from representation.
DTO	Data Transfer Object — a flat object used to carry data across process or service boundaries.
ERD	Entity Relationship Diagram — a model of persistent data entities and their relationships.
Facade	Structural design pattern providing a unified, simplified interface to a set of subsystem interfaces.
Factory Method	Creational design pattern that defers object instantiation to subclasses through an overridable creation method.
JWT	JSON Web Token — a signed, self-contained token used for stateless authentication.
Microservices	Architectural style decomposing a system into small, independently deployable services, each owning its data.
Observer	Behavioral design pattern in which subscribers (observers) are notified automatically of a subject's state changes.
PCI DSS	Payment Card Industry Data Security Standard — security requirements for handling cardholder data.
Proxy	Structural design pattern providing a surrogate (e.g., a cache) that controls access to a real object.
Singleton	Creational design pattern ensuring a class has exactly one instance with a global access point.
State	Behavioral design pattern that lets an object alter its behavior when its internal state changes.
Strategy	Behavioral design pattern that encapsulates interchangeable algorithms behind a common interface.
WebSocket	Full-duplex communication protocol over a single TCP connection, used here for live GPS push.

5. Design Body

This section presents the design of the Online Food Delivery System through the IEEE 1016 viewpoints. Each viewpoint addresses a specific set of design concerns held by the stakeholders identified in Section 5.1, and the rationale for selecting each viewpoint follows directly from those concerns.

5.1 Design Stakeholders and Their Concerns

An SDD shall address each identified design concern. The table below maps every design stakeholder of the Online Food Delivery System to their primary concerns and to the viewpoint(s) in this document that resolve them.

Design Stakeholder	Primary Design Concerns	Addressing Viewpoint(s)
Customer (end user)	Easy discovery, fast checkout, transparent live tracking, data privacy.	Context (5.2), Interface (5.5), Interaction (5.7)
Software Architect	Adherence to the microservices style; scalability, fault isolation, security segmentation.	Structure (5.3), Logical (5.6)
Developer	Clear class structure, correct pattern application, code that matches the UML.	Logical (5.6), Interaction (5.7), Algorithm (5.8)
Database Administrator	Well-normalized persistent data, integrity constraints, clear entity relationships.	Information (5.4)
QA Engineer	Testable flows, defined alternative and exception paths, deterministic state transitions.	Context (5.2), Interaction (5.7), Algorithm (5.8)
Admin / Operations	Restaurant and fleet management, monitoring, low-friction deployment per service.	Context (5.2), Structure (5.3)
Payment Compliance Officer	PCI DSS isolation; no card data stored on platform.	Structure (5.3), Logical (5.6 — M3)

5.2 Context Viewpoint (Use Case Diagram)

5.2.1 Design Concerns

The Context viewpoint depicts the services the design subject provides with reference to an explicit context defined by its actors. Its purpose is to identify the system's offered services (system features), the actors that interact with it, and the boundary between the system and its environment. This view answers the concerns of customers, admins, and QA engineers regarding what the system does and who uses it.

System features. The Online Food Delivery System offers seven primary features: register/login, browse restaurants & menu, add items to cart, place & confirm order, process payment, track delivery in real time, and manage restaurants & fleet (admin). The external actors are the Customer, Admin, Delivery Driver, the Stripe Payment Gateway, and the Email/Notification Service.

5.2.2 Design View (Use Case Diagram)

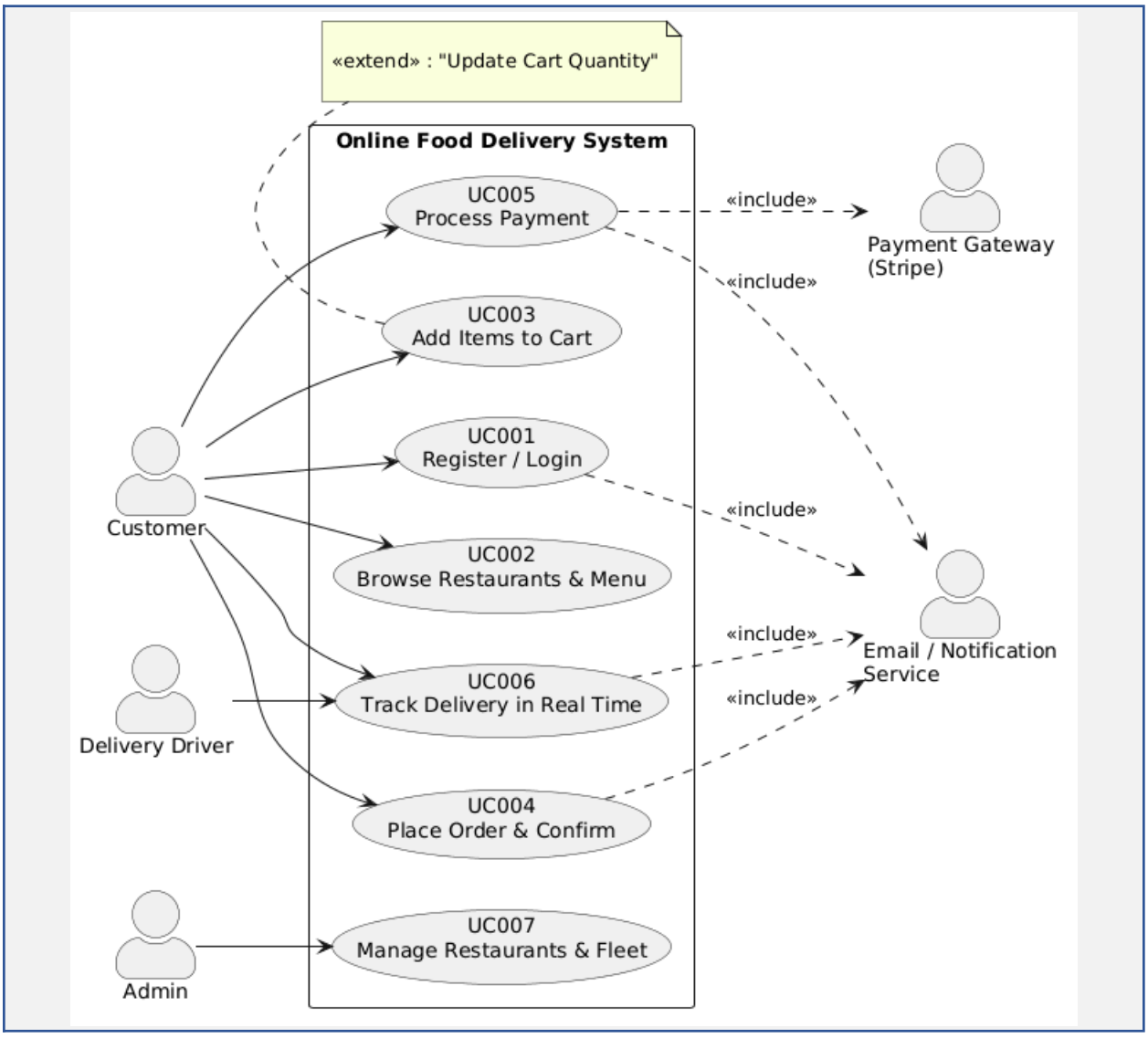


Figure 5.2.1 — Use Case Diagram for the Online Food Delivery System

Actors

Actor	Type	Description
Customer	Human (primary)	Browses restaurants, places orders, pays, and tracks deliveries.
Admin	Human	Manages restaurant listings and the delivery fleet; monitors analytics.
Delivery Driver	Human (external)	Fulfils active deliveries and updates GPS position.
Payment Gateway (Stripe)	External System	Tokenises cards, processes charges, and handles refunds.
Email / Notification Service	External System	Dispatches order confirmations and live status updates.

Use Case Specifications

Each primary use case is specified below. Alternative-flow scenarios are enumerated because they are realized as branches (alt frames) in the sequence diagrams of Section 5.7.

UC001 — Register / Login (Module 1)

Use case:	Register / Login
ID:	UC001
Actors:	Customer (primary), AuthService, UserRepository
Preconditions:	Customer has access to the web/mobile app. For login, the customer has a registered account; email format is valid. AuthService and UserDB are operational.
Flow of events:	1. Customer opens the Login / Register page. 2. Customer enters email and password. 3. System validates email format and password length (min 8). 4. AuthService.authenticate(email, password) is invoked. 5. UserRepository.findByEmail(email) returns the user record. 6. AuthService.verifyPasswordHash(input, hash) validates credentials. 7. AuthService.createSession(userId) records the session. 8. A signed JWT is issued and stored in an HttpOnly cookie. 9. Customer is redirected to the home page.
Postconditions:	Customer holds a valid JWT session token. Session recorded; for registration, a new user record is created.
Alternative flow 1:	Invalid credentials — verifyPasswordHash returns false; after 5 attempts the account locks for 15 minutes.
Alternative flow 2:	Account does not exist — registration form shown; AuthService.register(dto) creates the record, then resumes at step 8.
Exception flow:	Duplicate email on registration — a unique-constraint exception yields 'This email is already registered.'

UC002 — Browse Restaurants & Menu (Module 1)

Use case:	Browse Restaurants & Menu
ID:	UC002
Actors:	Customer (primary), RestaurantService (via RestaurantCacheProxy), RestaurantDB
Preconditions:	Customer is authenticated. Customer's location is available.

	RestaurantService and RestaurantDB are operational.
Flow of events:	1. Customer arrives at the home page and applies optional filters (cuisine, rating). 2. RestaurantCacheProxy.searchRestaurants(location, cuisine, rating) is invoked. 3. On a cache miss, the proxy calls RestaurantService.searchRestaurants(). 4. RestaurantService.queryRestaurants(filters) returns matching records. 5. Results are mapped to RestaurantDTO[] and cached. 6. Customer selects a restaurant; getMenuItems(restaurantId) returns the menu.
Postconditions:	A filtered restaurant list and the chosen menu are displayed. No data mutation occurs (read-only).
Alternative flow 1:	No restaurants found — empty result; system suggests widening the search radius.
Alternative flow 2:	Restaurant temporarily closed — isOpen = false; a 'Currently Closed' badge is shown.
Exception flow:	RestaurantDB unavailable — proxy serves the last cached result where possible; otherwise a retry message is shown.

UC003 / UC004 — Add to Cart & Place Order (Module 2)

Use case:	Add Items to Cart & Place Order
ID:	UC003 (add), UC004 (place)
Actors:	Customer (primary), CartService, InventoryService, CheckoutFacade, OrderService, NotificationService
Preconditions:	Customer is authenticated and viewing a menu. Selected item is available; restaurant is open. CartService and InventoryService are operational.
Flow of events:	1. CartService.addItem(customerId, menuItemId, qty) is invoked. 2. InventoryService.checkAvailability(menuItemId, qty) confirms stock. 3. CartService.createCartItem() persists the item; updateCartTotal() recalculates. 4. Customer proceeds to checkout; CheckoutFacade.checkout(cartId, address) orchestrates. 5. validateCart() then reserveInventory() returns a reservationId. 6. OrderService.createOrder() uses OrderBuilder to build the Order. 7. Order state is set to PENDING_PAYMENT; OrderSummaryDTO returned. 8. NotificationService.sendOrderConfirmation() is dispatched asynchronously.
Postconditions:	Order created with status PENDING_PAYMENT; inventory reserved. Cart marked CONVERTED; customer redirected to payment.
Alternative flow 1:	Duplicate item — existing CartItem quantity incremented instead of creating a duplicate.
Alternative flow 2:	Empty cart at checkout — customer redirected to browse with 'Your cart is empty.'
Exception flow:	InsufficientStockException at reservation — customer told which item is unavailable and must adjust the cart.

UC005 / UC006 — Process Payment & Track Delivery (Module 3)

Use case:	Process Payment & Track Delivery
ID:	UC005 (pay), UC006 (track)
Actors:	Customer (primary), PaymentService, Payment Gateway (Stripe), DeliveryService, TrackingService, Delivery Driver
Preconditions:	Order exists with status PENDING_PAYMENT. PaymentService and Payment Gateway are operational.

	For tracking: payment confirmed and a driver assigned.
Flow of events:	1. PaymentGatewayFactory.createGateway() returns the configured gateway. 2. PaymentService.processPayment(orderId, token, amount) validates the request. 3. StripeGatewayAdapter.chargeCard(token, amount, 'MYR') delegates to the external API. 4. On APPROVED, recordTransaction() persists the transaction; order set to CONFIRMED. 5. dispatchDelivery() is triggered asynchronously. 6. Customer opens tracking; TrackingService.subscribe(client) registers an observer. 7. Every 10s, broadcastLocation() pushes GPS updates over WebSocket.
Postconditions:	Transaction recorded as APPROVED; order CONFIRMED. Live driver location and ETA streamed until DELIVERED.
Alternative flow 1:	Payment declined — customer notified and prompted to try another method.
Alternative flow 2:	GPS signal lost — last known location shown; system attempts reconnection.
Exception flow:	Gateway timeout — transaction marked PENDING and reconciled; no double charge.

5.3 Structure Viewpoint (Architecture Style — Assignment 1)

5.3.1 Design Concerns

The Structure viewpoint documents the internal constituents and organization of the system in terms of its services and their interactions. It informs the architect and developer about how the major components (microservices) are partitioned, how they communicate, and how the architecture satisfies the non-functional requirements.

Selected architectural style: Microservices. Carried forward from Assignment 1, the system is decomposed into small, independently deployable services — each owning its data store and API surface. Services communicate via REST/HTTP for synchronous calls and an event bus for asynchronous operations (notifications, delivery dispatch), with WebSocket used for live GPS push.

Non-Functional Requirement	How Microservices Satisfies It
Scalability — 50,000 concurrent users at peak	RestaurantService auto-scales independently (e.g., 3→15 instances) without touching PaymentService.
Performance — search P99 < 300ms	RestaurantCacheProxy (Proxy pattern) serves hot queries from cache, cutting DB load.
Security — PCI DSS; no card data stored	PaymentService runs in an isolated PCI-scoped segment; StripeGatewayAdapter keeps card data off-platform.
Availability — 99.9% uptime	Fault isolation: a DeliveryService outage does not cascade to browsing or payment.
Maintainability — < 1 hour deploy per service	Each module maps to one independently ownable, independently deployable service.

5.3.2 Design View (Component / Architecture Diagram)

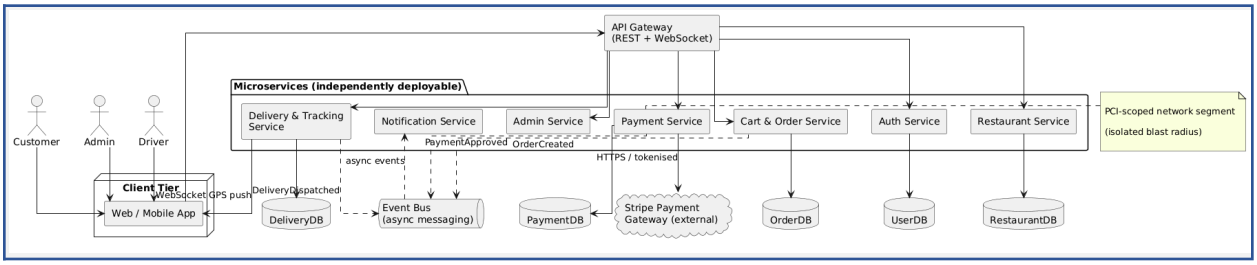


Figure 5.3.1 — Microservices Component Architecture for the Online Food Delivery System

5.4 Information Viewpoint (Database Design)

5.4.1 Design Concerns

The Information viewpoint applies because the system has substantial persistent data. It addresses how the information domain is transformed into data structures, how major entities are stored and related, and how integrity is preserved. In a microservices deployment each service owns its schema; the consolidated ERD below presents the logical data model spanning those service-owned stores.

5.4.2 Design View (ERD)

The major data entities are stored in a relational model comprising ten entities. They are listed alphabetically below, followed by the data dictionary and the Entity Relationship Diagram.

Entity Name	Description
CART	A customer's active shopping cart; aggregates cart items and a running subtotal.
CART_ITEM	A single menu item with quantity and unit price inside a cart.
DELIVERY	A delivery record linking an order to a driver, with live coordinates and ETA.
DRIVER	A delivery driver who fulfils deliveries.
MENU_ITEM	A purchasable dish offered by a restaurant.
ORDERS	A confirmed order converted from a cart, with status and total.
RESTAURANT	A restaurant listing that offers menu items.
TRANSACTION	A payment record for an order, with gateway reference and status.
USER	A registered customer or admin account.

Data Dictionary

Entity: USER

Attribute Name	Type	Description
user_id	BIGINT (PK)	Unique identifier for the user.
name	VARCHAR(120)	Full display name.
email	VARCHAR(160) (UQ)	Login identifier; unique.
password_hash	VARCHAR(255)	Salted hash of the password.
phone	VARCHAR(20)	Contact number.
role	ENUM	CUSTOMER or ADMIN.
created_at	TIMESTAMP	Account creation time.

Entity: ORDERS

Attribute Name	Type	Description
order_id	BIGINT (PK)	Unique identifier for the order.
user_id	BIGINT (FK)	Owning customer.
cart_id	BIGINT (FK)	Source cart that was converted.
delivery_address	VARCHAR(255)	Drop-off address.

Attribute Name	Type	Description
total_amount	DECIMAL(10,2)	Order grand total.
status	ENUM	PENDING_PAYMENT, CONFIRMED, OUT_FOR_DELIVERY, DELIVERED, CANCELLED.
created_at	TIMESTAMP	Order creation time.

Entity: TRANSACTION

Attribute Name	Type	Description
transaction_id	BIGINT (PK)	Unique identifier for the transaction.
order_id	BIGINT (FK)	Order being paid for.
gateway_ref	VARCHAR(120)	Reference returned by Stripe.
amount	DECIMAL(10,2)	Charged amount.
currency	CHAR(3)	ISO currency code (e.g., MYR).
status	ENUM	APPROVED, DECLINED, PENDING.
paid_at	TIMESTAMP	Settlement timestamp.

Entity: DELIVERY

Attribute Name	Type	Description
delivery_id	BIGINT (PK)	Unique identifier for the delivery.
order_id	BIGINT (FK)	Order being delivered.
driver_id	BIGINT (FK)	Assigned driver.
status	ENUM	PREPARING, OUT_FOR_DELIVERY, DELIVERED.
current_lat	DECIMAL(9,6)	Live latitude.
current_lng	DECIMAL(9,6)	Live longitude.
eta	TIMESTAMP	Estimated arrival time.

(CART, CART_ITEM, MENU_ITEM, RESTAURANT, and DRIVER follow the structure shown in the ERD.)

Entity Relationship Diagram (ERD)

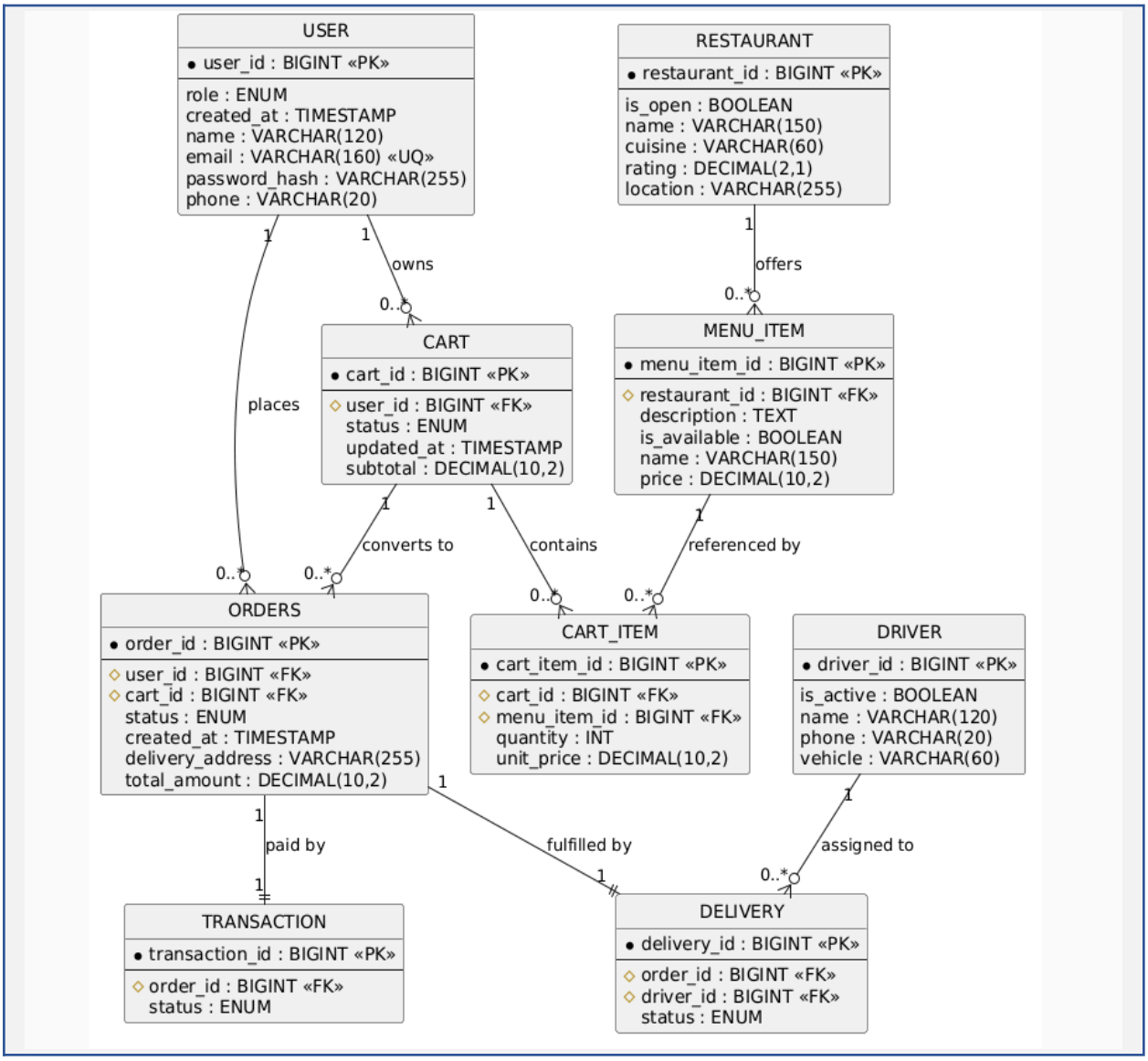


Figure 5.4.1 — Entity Relationship Diagram for the Online Food Delivery System

5.5 Interface Viewpoint (Interface Design)

5.5.1 Design Concerns

The Interface viewpoint provides designers, programmers, and testers the means to correctly use the services the system offers. It covers the user interfaces, hardware interfaces, software interfaces, and communication interfaces through which the system interacts with its environment.

User Interfaces

The system exposes a responsive web and mobile UI. The logical characteristics of the key screens are: a Login/Register screen; a Restaurant Browse screen with a search bar, filter controls, and a results list (name, rating, ETA); a Cart screen with line items, quantity controls, and a price summary; a Payment screen with a PCI-compliant tokenised card iframe; and a live Tracking screen with a map, driver marker, status, and ETA. Interface do's and don'ts: always show clear feedback on add-to-cart and payment; never expose raw card data to the application layer; keep primary actions (Checkout, Pay Now) prominent and single-tap.

Hardware Interfaces

The client runs on standard consumer hardware (smartphones and desktop browsers) and requires GPS/location services on the customer device for delivery radius detection and on the driver device for live position reporting. No specialized hardware is required on the client side; server-side services run on containerized cloud compute.

Software Interfaces

Interface	Detail
Name / Mnemonic	Stripe Payment Gateway API (STRIPE)
Version / Source	Stripe REST API v1 (external, https://api.stripe.com)
Purpose	Tokenises cards and processes charges/refunds so the platform never stores card data (PCI DSS).
Message content & format	HTTPS/JSON requests (createCharge with token, amount, currency); JSON responses with transaction id and status. Adapted internally by StripeGatewayAdapter.

Other software interfaces: the Email/Notification Service (SMTP/Push provider) for confirmations, and a relational DBMS per service for persistence.

Communication Interfaces

Inter-service communication uses REST over HTTPS for synchronous calls and an asynchronous event bus (publish/subscribe) for OrderCreated, PaymentApproved, and DeliveryDispatched events. Live delivery tracking uses the WebSocket protocol for low-latency, server-pushed GPS updates to the customer's browser, avoiding polling overhead.

5.5.2 Design View (Wireframes)



Figure 5.5.1 — Key Screen Wireframes (Login, Browse, Cart, Tracking)

5.6 Logical Viewpoint (Creational, Structural & Behavioral Patterns — Assignment 2)

5.6.1 Design Concerns

The Logical viewpoint elaborates the implemented types — classes and interfaces — and their static relationships within each module. The Assignment 2 mandate is to evolve each module by injecting exactly one creational, one structural, and one behavioral design pattern, then to generate Java code translated directly from the resulting UML. Every attribute and method that appears in the code below also appears in the class diagrams, and vice versa, preserving strict architectural coherence.

The table below is the Synthesis Matrix — the pattern triad selected for each module. Each pattern was chosen to fit logic that already existed in the Assignment 1 design, so the patterns integrate seamlessly rather than being bolted on.

Module / Owner	Creational	Structural	Behavioral
Module 1 **Mohammed Alsakkaf**	Singleton (ConfigurationManager)	Proxy (RestaurantCacheProxy)	Strategy (AuthenticationStrategy)
Module 2 **Anas Ahmed Hisham**	Builder (OrderBuilder)	Facade (CheckoutFacade)	State (OrderState)
Module 3 **Imtiaj Ahmed**	Factory Method (PaymentGatewayFactory)	Adapter (StripeGatewayAdapter)	Observer (TrackingService)

Code rule: the Java samples are direct structural translations of the UML. If an attribute or method exists in the code, it exists in the class diagram; if it is in the diagram, it is accounted for in the implementation. Snippets are commented heavily and mapped back to the sequence and class diagrams.

5.6.2 Module 1 — User Auth & Restaurant Browsing | Mohammed Alsakkaf

Module 1 applies the Singleton pattern for global configuration, the Strategy pattern to make the authentication algorithm interchangeable, and the Proxy pattern to add transparent caching in front of restaurant search.

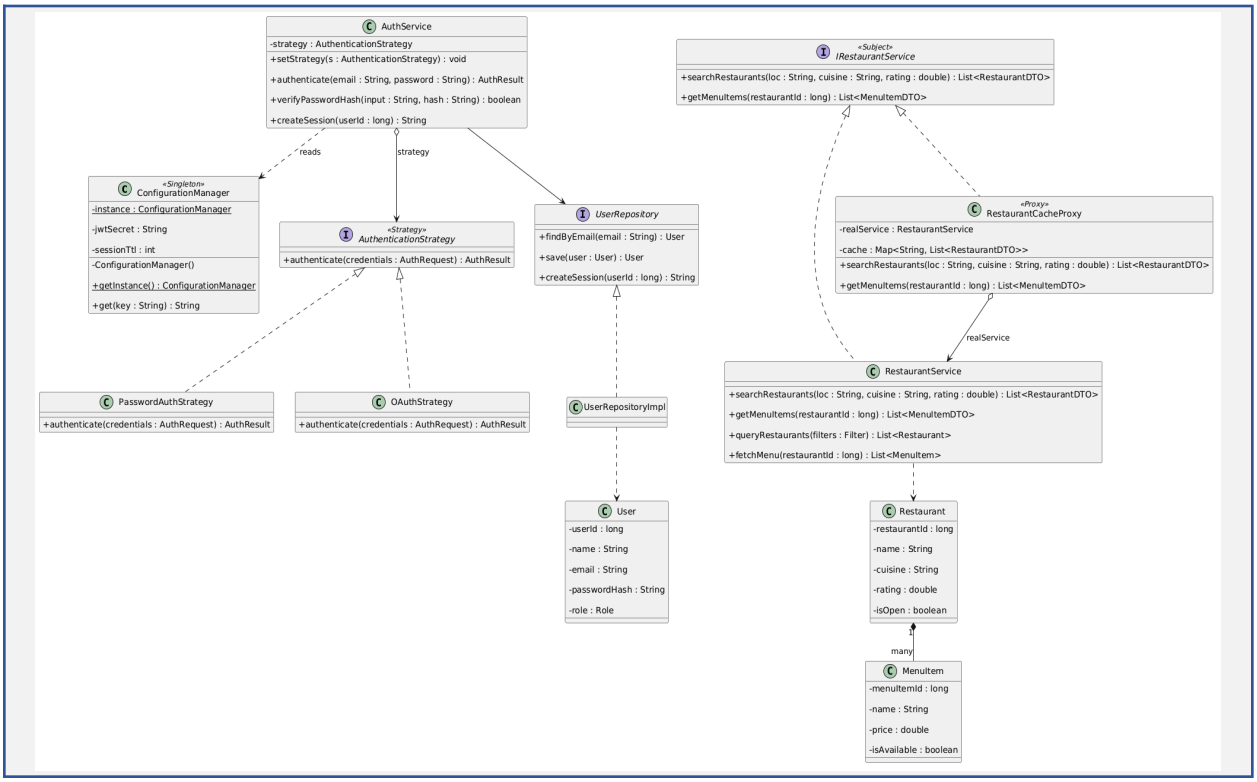


Figure 5.6.1 — Class Diagram with Singleton + Proxy + Strategy — Module 1 — Mohammed Alsakkaf

(a) Creational Pattern — Singleton (ConfigurationManager)

ConfigurationManager holds shared settings (JWT secret, session TTL) and must exist exactly once across the AuthService. The Singleton guarantees a single instance with lazy, thread-safe initialization.

```
// Singleton – single shared configuration across the Auth Service
public final class ConfigurationManager {
    private static volatile ConfigurationManager instance;   // UML:
-instance:ConfigurationManager
    private final Map<String, String> settings = new HashMap<>();

    private ConfigurationManager() {                         // private ctor blocks external
'new'
        settings.put("jwtSecret", System.getenv("JWT_SECRET"));
        settings.put("sessionTtl", "3600");
    }

    public static ConfigurationManager getInstance() {       // UML: +getInstance()
        if (instance == null) {                              // first check (no lock)
            synchronized (ConfigurationManager.class) {
                if (instance == null) {                      // double-checked locking
                    instance = new ConfigurationManager();
                }
            }
        }
```

```
        return instance;
    }

    public String get(String key) { return settings.get(key); } // UML: +get(key)
}
```

(b) Behavioral Pattern — Strategy (AuthenticationStrategy)

AuthService delegates the act of verifying a user to an interchangeable AuthenticationStrategy, so password login and OAuth login can be swapped without changing AuthService.

```
// Strategy – interchangeable authentication algorithms
public interface AuthenticationStrategy {                    // UML: <<Strategy>>
    AuthResult authenticate(AuthRequest credentials);
}

public class PasswordAuthStrategy implements AuthenticationStrategy {
    private final UserRepository userRepo;
    public PasswordAuthStrategy(UserRepository repo) { this.userRepo = repo; }

    @Override
    public AuthResult authenticate(AuthRequest c) {          // Seq 5.7.1: STRAT.authenticate()
        User user = userRepo.findByEmail(c.getEmail());      // Seq: findByEmail()
        if (user == null) return AuthResult.fail("NO_ACCOUNT");
        boolean ok = verifyPasswordHash(c.getPassword(), user.getPasswordHash());
        return ok ? AuthResult.success(user) : AuthResult.fail("BAD_CREDENTIALS");
    }
    private boolean verifyPasswordHash(String input, String hash) {
        return BCrypt.checkpw(input, hash);                  // UML:
AuthService.verifyPasswordHash()
    }
}

public class AuthService {
    private AuthenticationStrategy strategy;                 // UML: -strategy
    public void setStrategy(AuthenticationStrategy s) { this.strategy = s; }

    public AuthResult authenticate(String email, String password) {
        String secret = ConfigurationManager.getInstance().get("jwtSecret"); // Singleton use
        AuthResult r = strategy.authenticate(new AuthRequest(email, password));
        if (r.isSuccess()) createSession(r.getUser().getUserId());
        return r;
    }
    public String createSession(long userId) {               // UML: +createSession()
        return JwtUtil.sign(userId, /*ttl*/ 3600);
    }
}
```

(c) Structural Pattern — Proxy (RestaurantCacheProxy)

RestaurantCacheProxy implements the same IRestaurantService interface as the real service and stands in front of it, returning cached results on a hit and delegating on a miss. This realizes the Assignment 1 'cache layer' that keeps search P99 under 300ms.

```
// Proxy – caching surrogate sharing the subject's interface
public interface IRestaurantService {                        // UML: <<Subject>>
    List<RestaurantDTO> searchRestaurants(String loc, String cuisine, double rating);
    List<MenuItemDTO> getMenuItems(long restaurantId);
}

public class RestaurantCacheProxy implements IRestaurantService {  // UML: <<Proxy>>
```

```
    private final RestaurantService real;                    // UML: -realService
    private final Map<String, List<RestaurantDTO>> cache = new ConcurrentHashMap<>();
    public RestaurantCacheProxy(RestaurantService real) { this.real = real; }

    @Override
    public List<RestaurantDTO> searchRestaurants(String loc, String cuisine, double rating) {
        String key = loc + "|" + cuisine + "|" + rating;
        if (cache.containsKey(key)) return cache.get(key);   // Seq 5.7.1: cache hit
        List<RestaurantDTO> result = real.searchRestaurants(loc, cuisine, rating); // miss ->
delegate
        cache.put(key, result);                              // populate cache
        return result;
    }
    @Override
    public List<MenuItemDTO> getMenuItems(long restaurantId) {
        return real.getMenuItems(restaurantId);              // delegate (menus change rarely)
    }
}
```

Coherence Validation — Module 1

Sequence Message (Fig 5.7.1)	Class / Pattern Method (Fig 5.6.1)	Match
getInstance().get("jwtSecret")	ConfigurationManager.getInstance/get() — Singleton	✓
strategy.authenticate(credentials)	AuthenticationStrategy.authenticate() — Strategy	✓
findByEmail(email)	UserRepository.findByEmail()	✓
verifyPasswordHash(input, hash)	AuthService.verifyPasswordHash()	✓
createSession(userId)	AuthService.createSession()	✓
searchRestaurants(loc, cuisine, rating)	RestaurantCacheProxy.searchRestaurants() — Proxy	✓
real.searchRestaurants(...)	RestaurantService.searchRestaurants()	✓

5.6.3 Module 2 — Shopping Cart & Order Placement | Anas Ahmed Hisham

Module 2 applies the Builder pattern to assemble complex Order objects, the Facade pattern to give the WebApp one simple checkout entry point over three subsystems, and the State pattern to govern the order lifecycle.

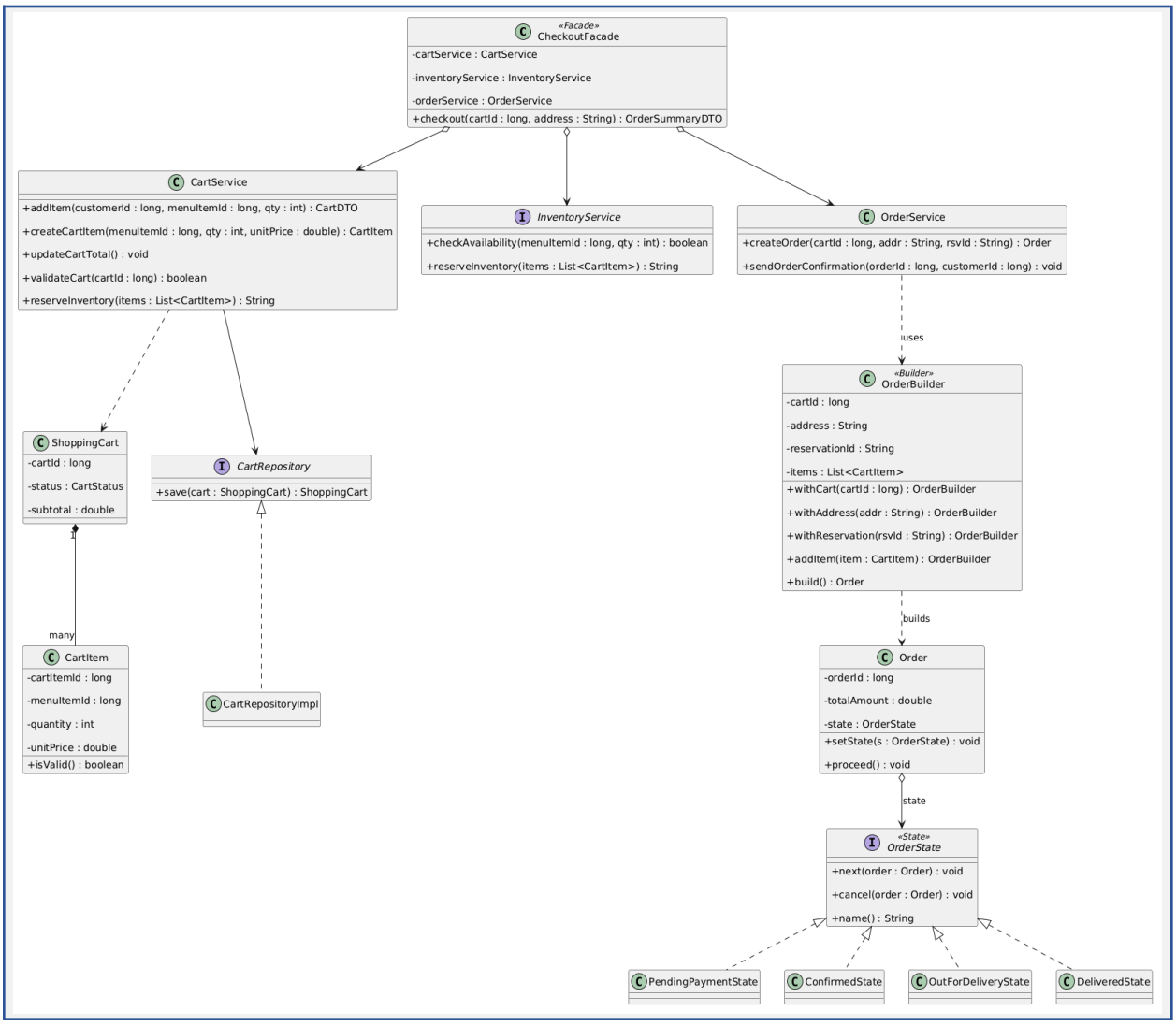


Figure 5.6.2 — Class Diagram with Builder + Facade + State — Module 2 — Anas Ahmed Hisham

(a) Creational Pattern — Builder (OrderBuilder)

An Order requires a cart reference, address, reservation id, and item list. OrderBuilder assembles it step by step with a fluent API, keeping construction logic out of Order itself.

```
// Builder — step-by-step construction of a complex Order
public class OrderBuilder {                                    // UML: <<Builder>>
    private long cartId;
    private String address;
    private String reservationId;
    private final List<CartItem> items = new ArrayList<>();

    public OrderBuilder withCart(long cartId)       { this.cartId = cartId; return this; }
```

```
    public OrderBuilder withAddress(String addr)     { this.address = addr; return this; }
    public OrderBuilder withReservation(String rsv)  { this.reservationId = rsv; return this; }
    public OrderBuilder addItem(CartItem item)       { this.items.add(item); return this; }

    public Order build() {                                    // UML: +build():Order
        Order order = new Order(cartId, address, reservationId, items);
        order.setState(new PendingPaymentState());           // initial state (State pattern)
        return order;
    }
}
```

(b) Structural Pattern — Facade (CheckoutFacade)

CheckoutFacade hides the three-step orchestration (validate cart, reserve inventory, create order) behind a single checkout() call, decoupling the WebApp from the cart, inventory, and order subsystems.

```
// Facade – one simple entry point over three subsystems
public class CheckoutFacade {                                 // UML: <<Facade>>
    private final CartService cartService;
    private final InventoryService inventoryService;
    private final OrderService orderService;

    public CheckoutFacade(CartService c, InventoryService i, OrderService o) {
        this.cartService = c; this.inventoryService = i; this.orderService = o;
    }

    public OrderSummaryDTO checkout(long cartId, String address) { // Seq 5.7.2:
FACADE.checkout()
        if (!cartService.validateCart(cartId))               // Seq: validateCart()
            throw new EmptyCartException(cartId);
        String reservationId = inventoryService.reserveInventory(cartService.getItems(cartId));
// reserve
        Order order = orderService.createOrder(cartId, address, reservationId);  // create
        return OrderSummaryDTO.from(order);
    }
}
```

(c) Behavioral Pattern — State (OrderState)

Order behavior depends on its lifecycle stage. The State pattern moves the transition rules into state classes, so Order.proceed() does the right thing without large conditionals.

```
// State – order changes behavior as its state changes
public interface OrderState {                                 // UML: <<State>>
    void next(Order order);
    void cancel(Order order);
    String name();
}

public class PendingPaymentState implements OrderState {
    public void next(Order o)   { o.setState(new ConfirmedState()); }   // on payment approved
    public void cancel(Order o) { o.setState(new CancelledState()); }   // on failure/timeout
    public String name()        { return "PENDING_PAYMENT"; }
}
public class ConfirmedState implements OrderState {
    public void next(Order o)   { o.setState(new OutForDeliveryState()); }
    public void cancel(Order o) { throw new IllegalStateException("Confirmed order cannot
cancel"); }
    public String name()        { return "CONFIRMED"; }
}
```

```
public class Order {
    private OrderState state;                                  // UML: -state:OrderState
    public void setState(OrderState s) { this.state = s; }     // UML: +setState()
    public void proceed()  { state.next(this); }              // delegates to current state
    public String status() { return state.name(); }
}
```

Coherence Validation — Module 2

Sequence Message (Fig 5.7.2)	Class / Pattern Method (Fig 5.6.2)	Match
checkout(cartId, address)	CheckoutFacade.checkout() — Facade	✓
validateCart(cartId)	CartService.validateCart()	✓
reserveInventory(items)	InventoryService.reserveInventory()	✓
createOrder(cartId, addr, rsvId)	OrderService.createOrder()	✓
withCart().withAddress().build()	OrderBuilder.build() — Builder	✓
setState(PendingPaymentState)	Order.setState() — State	✓
sendOrderConfirmation(orderId, custId)	OrderService.sendOrderConfirmation()	✓

5.6.4 Module 3 — Payment Processing & Delivery Tracking | Imtiaj Ahmed

Module 3 applies the Factory Method pattern to create the correct payment gateway, the Adapter pattern to wrap the external Stripe API behind the internal PaymentGateway interface, and the Observer pattern to push live GPS updates to subscribers.

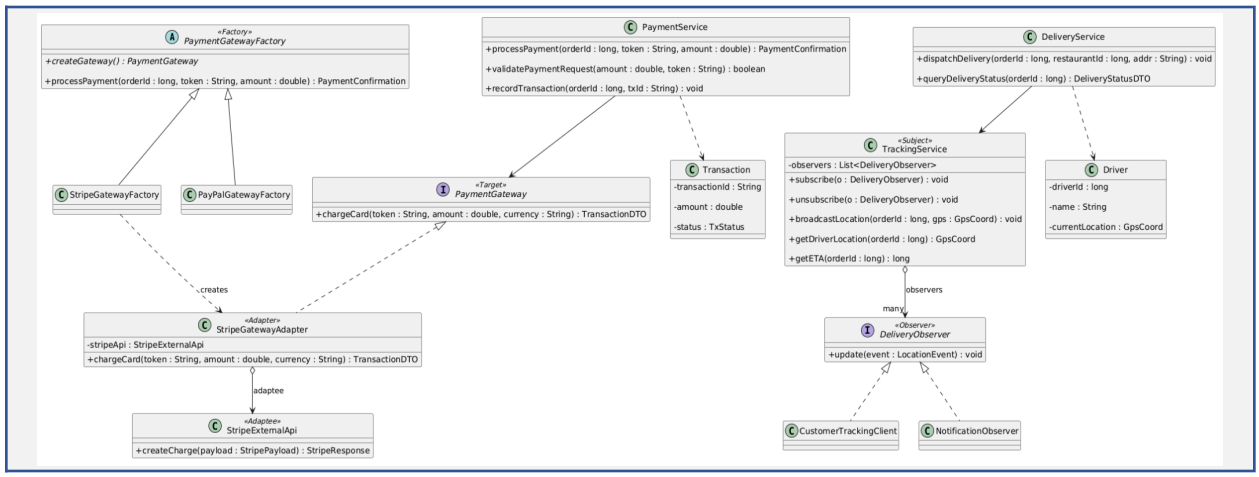


Figure 5.6.3 — Class Diagram with Factory Method + Adapter + Observer — Module 3 — Imtiaj Ahmed

(a) Creational Pattern — Factory Method (PaymentGatewayFactory)

Different providers (Stripe, PayPal) require different gateway objects. The Factory Method defers instantiation to subclasses, so PaymentService stays agnostic of the concrete provider.

```
// Factory Method — subclasses decide which gateway to instantiate
public abstract class PaymentGatewayFactory {                  // UML: <<Factory>>
    protected abstract PaymentGateway createGateway();         // factory method (overridden)

    public PaymentConfirmation processPayment(long orderId, String token, double amount) {
        PaymentGateway gateway = createGateway();              // Seq 5.7.3: createGateway()
        TransactionDTO tx = gateway.chargeCard(token, amount, "MYR");
        return new PaymentConfirmation(orderId, tx);
    }
}

public class StripeGatewayFactory extends PaymentGatewayFactory {
    @Override
    protected PaymentGateway createGateway() {
        return new StripeGatewayAdapter(new StripeExternalApi()); // returns adapter (below)
    }
}
```

(b) Structural Pattern — Adapter (StripeGatewayAdapter)

The internal code expects PaymentGateway.chargeCard(...), but Stripe exposes createCharge(payload). StripeGatewayAdapter converts one interface into the other, keeping card handling off-platform (PCI DSS).

```
// Adapter — converts Stripe's API to our PaymentGateway interface
public interface PaymentGateway {                              // UML: <<Target>>
    TransactionDTO chargeCard(String token, double amount, String currency);
}

public class StripeExternalApi {                               // UML: <<Adaptee>> (3rd-party)
```

```
    public StripeResponse createCharge(StripePayload payload) { /* HTTPS to Stripe */ return new
StripeResponse(); }
}

public class StripeGatewayAdapter implements PaymentGateway { // UML: <<Adapter>>
    private final StripeExternalApi stripeApi;               // UML: -stripeApi (adaptee)
    public StripeGatewayAdapter(StripeExternalApi api) { this.stripeApi = api; }

    @Override
    public TransactionDTO chargeCard(String token, double amount, String currency) {
        StripePayload payload = new StripePayload(token, (long)(amount * 100), currency); //
adapt request
        StripeResponse resp = stripeApi.createCharge(payload);          // call adaptee
        return new TransactionDTO(resp.getId(), resp.getStatus());      // adapt response
    }
}
```

(c) Behavioral Pattern — Observer (TrackingService)

Multiple parties (the customer's browser, the notification service) must react to each GPS update. TrackingService is the Subject; observers subscribe and are pushed updates over WebSocket every 10 seconds.

```
// Observer — subscribers are notified of every location change
public interface DeliveryObserver {                          // UML: <<Observer>>
    void update(LocationEvent event);
}

public class CustomerTrackingClient implements DeliveryObserver {
    public void update(LocationEvent e) {                    // Seq 5.7.3: WebSocket push
        WebSocket.push(e.getOrderId(), e.getGps(), e.getEta()); // animate marker on client
    }
}

public class TrackingService {                               // UML: <<Subject>>
    private final List<DeliveryObserver> observers = new CopyOnWriteArrayList<>(); // UML:
-observers
    public void subscribe(DeliveryObserver o)   { observers.add(o); }    // UML: +subscribe()
    public void unsubscribe(DeliveryObserver o) { observers.remove(o); }

    public void broadcastLocation(long orderId, GpsCoord gps) {          // UML:
+broadcastLocation()
        long eta = getETA(orderId);
        LocationEvent event = new LocationEvent(orderId, gps, eta);
        for (DeliveryObserver o : observers) o.update(event);            // notify all
    }
    public GpsCoord getDriverLocation(long orderId) { /* poll driver app */ return new
GpsCoord(); }
    public long getETA(long orderId) { /* compute ETA */ return 12; }
}
```

Coherence Validation — Module 3

Sequence Message (Fig 5.7.3)	Class / Pattern Method (Fig 5.6.3)	Match
createGateway()	PaymentGatewayFactory.createGateway() — Factory Method	✓
processPayment(orderId, token, amount)	PaymentService.processPayment()	✓
validatePaymentRequest(amount, token)	PaymentService.validatePaymentRequest()	✓

Sequence Message (Fig 5.7.3)	Class / Pattern Method (Fig 5.6.3)	Match
chargeCard(token, amount, 'MYR')	StripeGatewayAdapter.chargeCard() — Adapter	✓
createCharge(payload)	StripeExternalApi.createCharge() (adaptee)	✓
recordTransaction(orderId, txId)	PaymentService.recordTransaction()	✓
subscribe(client)	TrackingService.subscribe() — Observer	✓
update(LocationEvent)	DeliveryObserver.update() — Observer	✓
broadcastLocation(orderId, gps)	TrackingService.broadcastLocation()	✓

5.7 Interaction Viewpoint (Sequence Diagrams by Module)

5.7.1 Design Concerns

The Interaction viewpoint defines strategies for interaction among entities — why, where, how, and at what level actions occur. The functionalities of the system are conveyed through sequence diagrams that show the exact ordering of messages between objects during a scenario. Each diagram below incorporates the patterns introduced in Section 5.6, so the runtime collaboration matches the static structure.

5.7.2 Design View (Sequence Diagrams)

a) SD001 — Login & Browse Restaurants (Strategy + Proxy) — Module 1

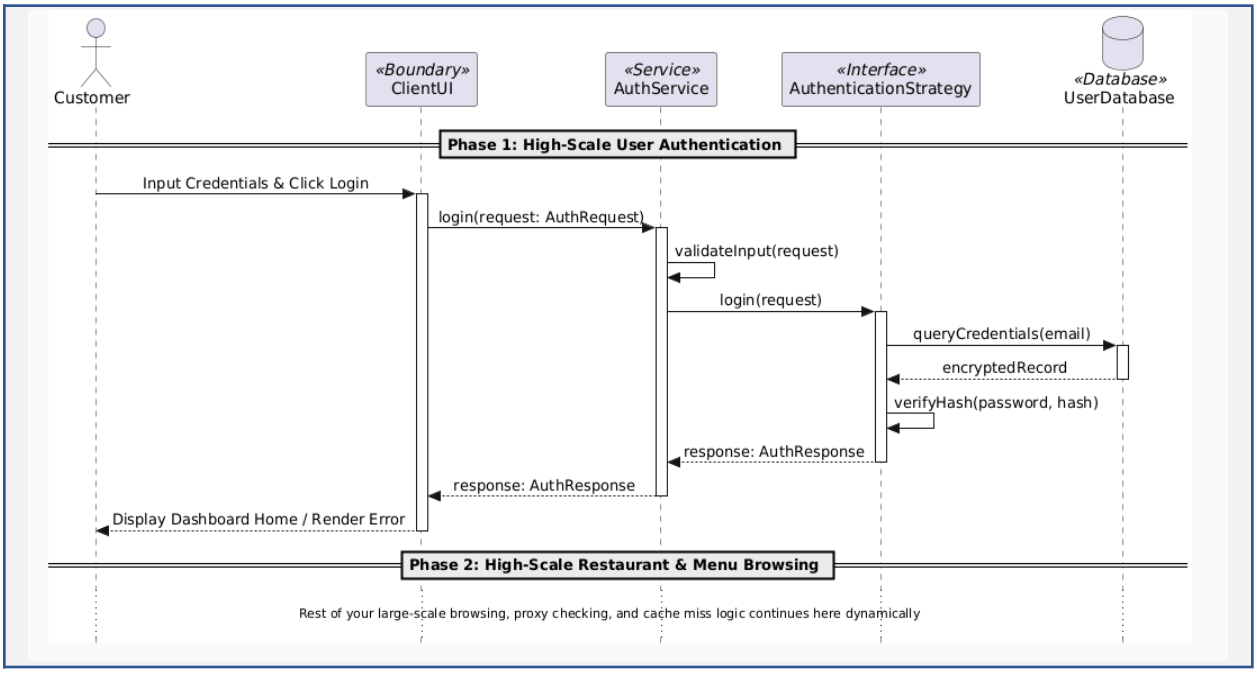


Figure 5.7.1 — Sequence Diagram: Login & Browse — Module 1 — Mohammed Alsakkaf

The login path runs Customer → WebApp → AuthService, which reads the JWT secret from the ConfigurationManager Singleton and delegates verification to the PasswordAuthStrategy. The browse path routes through the RestaurantCacheProxy, whose alt frame distinguishes a cache hit from a miss that delegates to the real RestaurantService.

b) SD002 — Add to Cart & Place Order (Facade + Builder + State) — Module 2

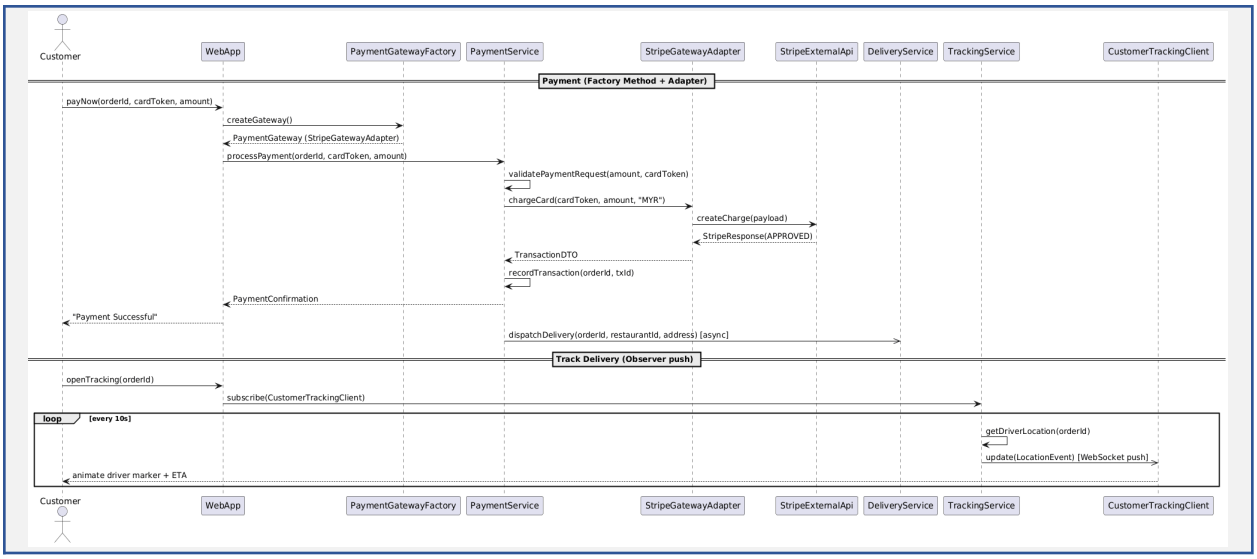


Figure 5.7.2 — Sequence Diagram: Add to Cart & Place Order — Module 2 — Anas Ahmed Hisham

The CheckoutFacade coordinates validateCart → reserveInventory → createOrder; the OrderBuilder assembles the Order, which is initialized to the PendingPaymentState. The asynchronous order-confirmation notification and the InsufficientStockException alternative flow are shown as an async message and an alt frame respectively.

c) SD003 — Process Payment & Track Delivery (Factory + Adapter + Observer) — Module 3

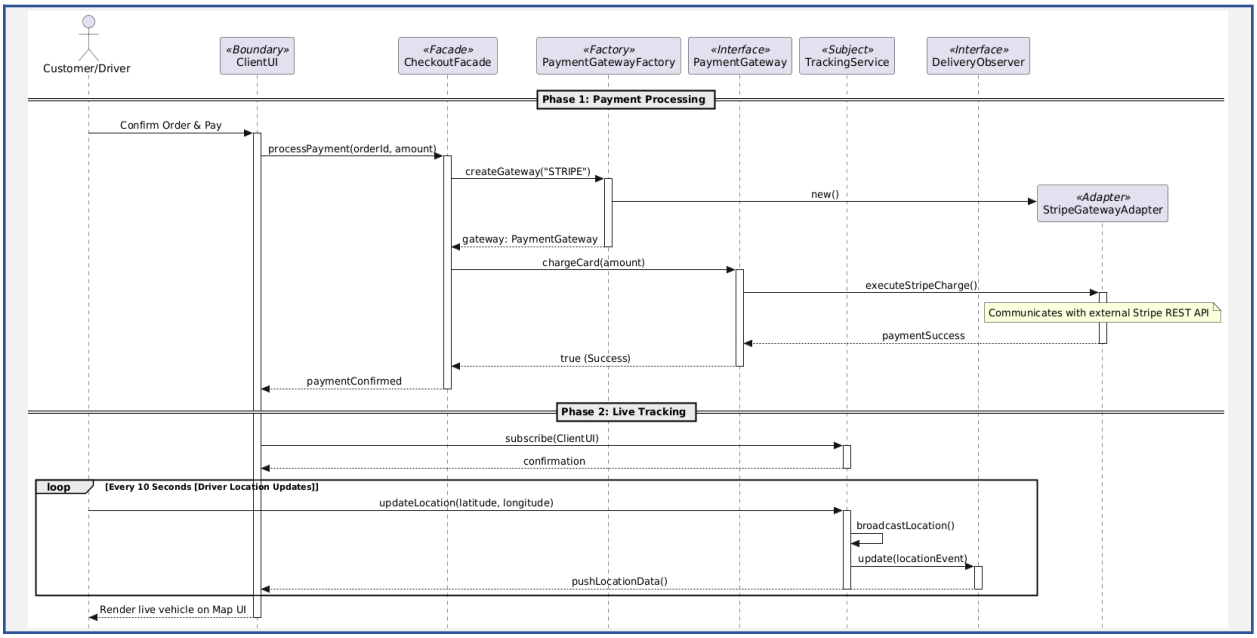


Figure 5.7.3 — Sequence Diagram: Process Payment & Track Delivery — Module 3 — Imtiaj Ahmed

The PaymentGatewayFactory yields a StripeGatewayAdapter, which translates chargeCard into Stripe's createCharge. After approval, delivery dispatch is asynchronous, and the tracking loop shows the Observer push: the CustomerTrackingClient receives update() events over WebSocket every 10 seconds.

5.8 Algorithm Viewpoint (Construction Design for Complex Processing)

5.8.1 Design Concerns

The Algorithm viewpoint provides the detailed processing logic of the system's most complex operation — the end-to-end transition from checkout through payment to delivery completion. It gives programmers and analysts the time-ordered decision logic and the order's state transitions prior to implementation.

5.8.2 Design View (Activity & State Diagrams)

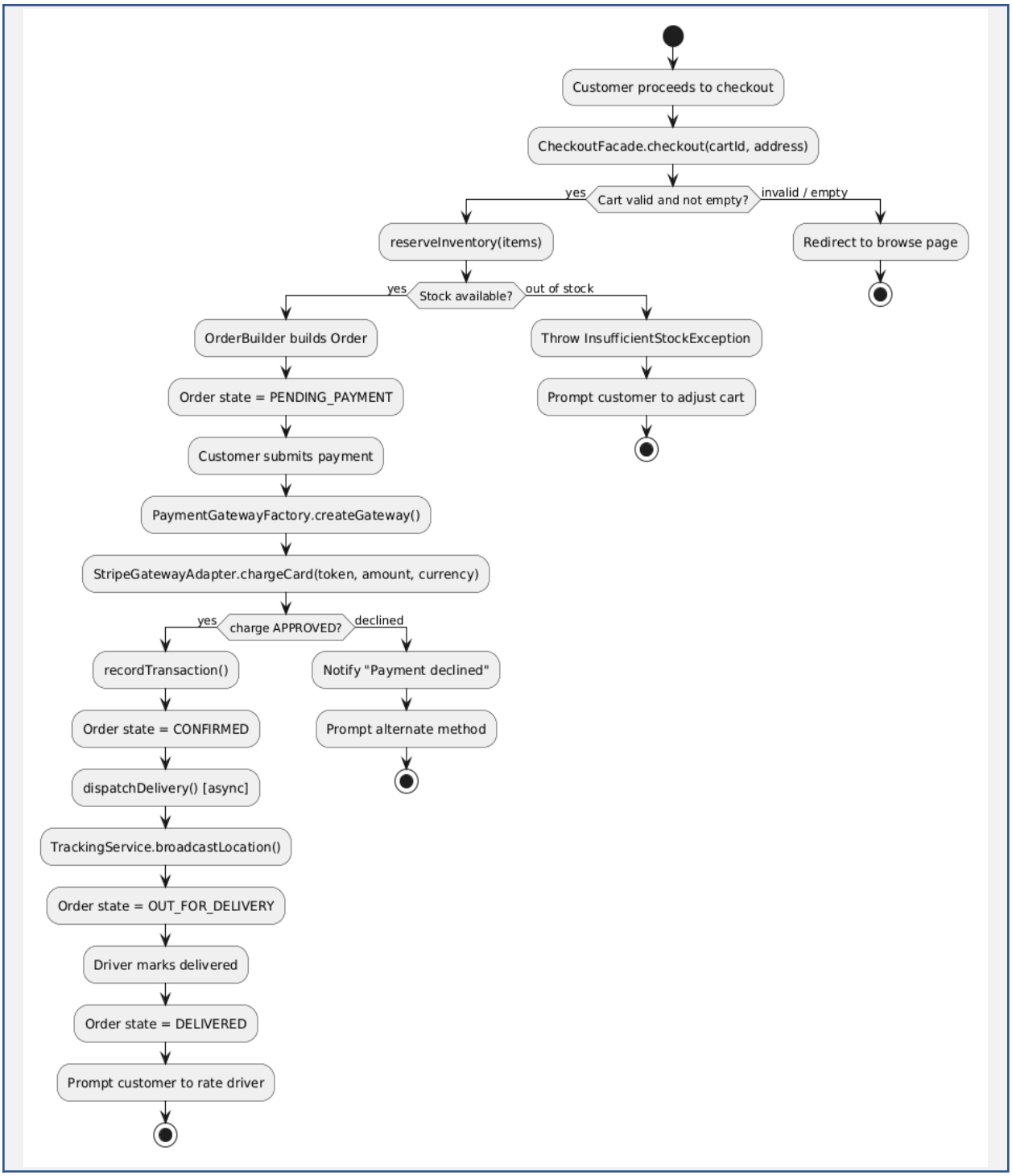


Figure 5.8.1 — Activity Diagram: End-to-End Order to Delivery

The activity diagram captures the principal decision points: cart validity, stock availability, and charge approval. Each failure branch maps to an alternative or exception flow defined in the use case specifications (Section 5.2).

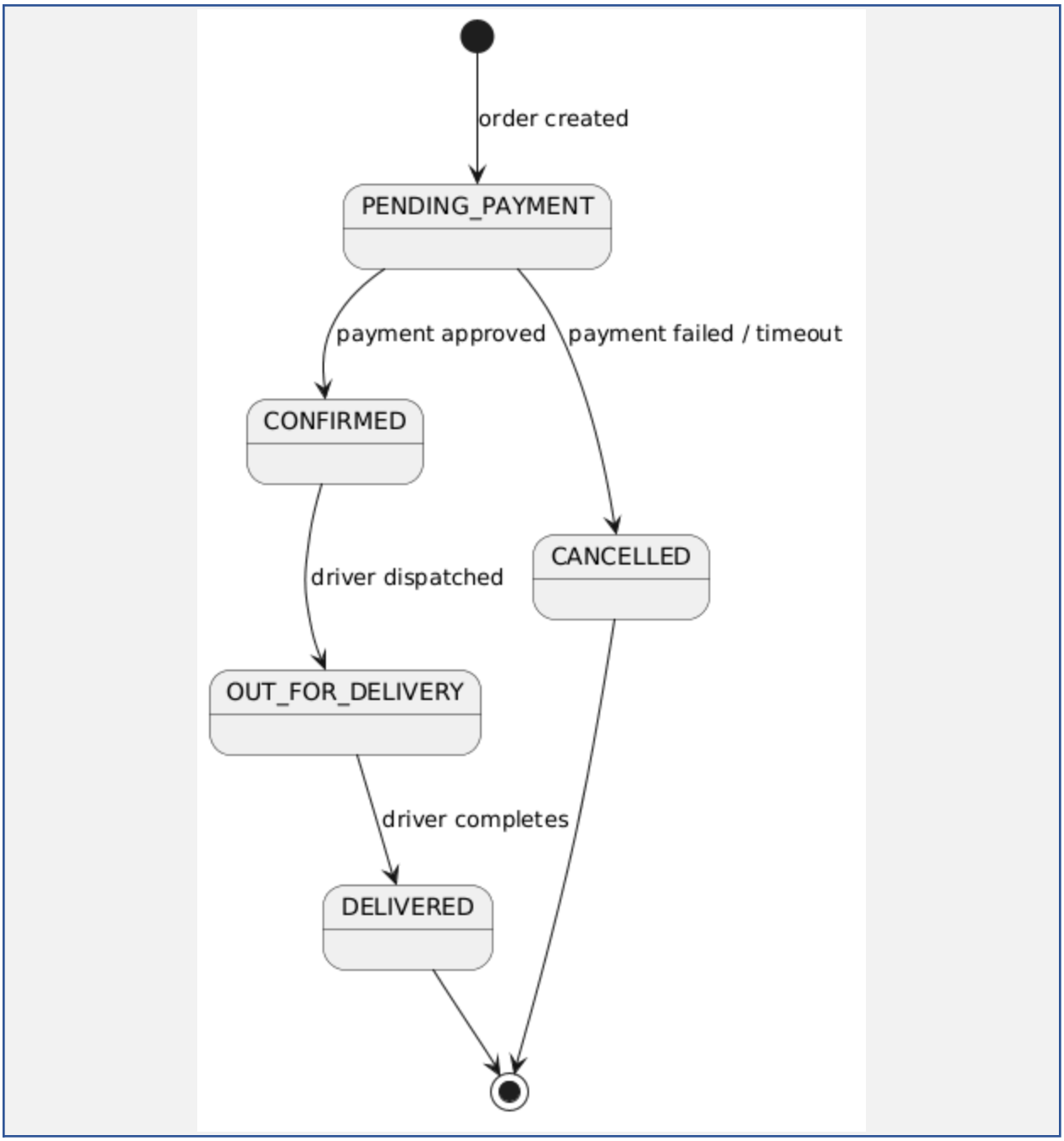


Figure 5.8.2 — State Machine Diagram: Order Lifecycle (realized by the State pattern, §5.6.3)

The state machine documents the legal transitions enforced by the State pattern: PENDING_PAYMENT → CONFIRMED → OUT_FOR_DELIVERY → DELIVERED, with PENDING_PAYMENT → CANCELLED as the only cancellation path. Illegal transitions (e.g., cancelling a confirmed order) are rejected by the corresponding state class.

6. User Interface Design

This section consolidates the user-facing design described in the Interface viewpoint (Section 5.5). The system follows a clean, mobile-first visual language with a single prominent primary action per screen, persistent cart visibility, and continuous status feedback during payment and delivery.

Screen	Key Elements	Primary Action
Login / Register	Email and password fields; toggle to registration; forgot-password link.	Log In
Browse Restaurants	Search bar, filter chips (cuisine, rating, ETA), result cards with rating and ETA.	View Menu
Menu	Items grouped by category with price, description, and dietary labels.	Add to Cart
Cart	Line items with quantity steppers, subtotal, delivery fee, and total.	Proceed to Checkout
Payment	Tokenised PCI-compliant card iframe; order summary; secure-padlock indicator.	Pay Now
Tracking	Live map with driver marker, status banner, ETA countdown, contact-driver button.	Contact Driver

Refer to Figure 5.5.1 for the wireframes of the Login, Browse, Cart, and Tracking screens.

7. Appendices

The appendices provide supporting detail that aids understanding of this Software Design Description.

7.1 Appendix A — Module Assignment & Pattern Matrix

Member	Module	Patterns Applied	Sections
Mohammed Alsakkaf	M1 — User Auth & Restaurant Browsing	Singleton, Proxy, Strategy	5.6.2, 5.7.1
Anas Ahmed Hisham	M2 — Shopping Cart & Order Placement	Builder, Facade, State	5.6.3, 5.7.2
Imtiaj Ahmed	M3 — Payment Processing & Delivery Tracking	Factory Method, Adapter, Observer	5.6.4, 5.7.3

7.2 Appendix B — Group Work Contributions

Name	Role	Contribution	%
Mohammed Alsakkaf	Lead Architect & Project Manager	Module 1 patterns & code, document consolidation, coherence validation, ERD.	44%
Anas Ahmed Hisham	System Analyst	Module 2 patterns & code, checkout flow, order state machine.	28%
Imtiaj Ahmed	Interface Designer	Module 3 patterns & code, payment adapter, tracking observer, wireframes.	28%

7.3 Appendix C — Final Quality Assurance Checklist

Requirement	Status
Each module applies one creational, one structural, and one behavioral pattern.	✓ Yes
Java code samples are direct translations of the UML class diagrams.	✓ Yes
Every sequence message maps to a class method (coherence tables 5.6.2–5.6.4).	✓ Yes
Use case alternative flows are realized as branches in the sequence diagrams.	✓ Yes
Information, Interface, and Algorithm viewpoints added per IEEE 1016.	✓ Yes
Individual work is explicitly labeled with each member's name and module.	✓ Yes
Microservices architectural style carried forward and justified against NFRs.	✓ Yes
Diagram placeholders provided for all required figures.	✓ Yes